SPPNet: An Approach For Real-Time Encrypted Traffic Classification Using Deep Learning

Fabien Meslet-Millet
IRIT/Toulouse INP-ENSEEIHT
University of Toulouse, France
firstname.lastname@irit.fr

Emmanuel Chaput
IRIT/Toulouse INP-ENSEEIHT
University of Toulouse, France
firstname.lastname@toulouse-inp.fr

Sandrine Mouysset
IRIT/UPS
University of Toulouse, France
firstname.lastname@irit.fr

GLOBECOM 2021 - 2021 IEEE Global Communications Conference | 978-1-7281-8104-2/21/$31.00 ©2021 IEEE | DOI: 10.1109/GLOBECOM46510.2021.9686037

***Abstract*—Data flow management has become a key network activity, strengthening the need for efficient data flow classification tools. However, pervasive encryption of communication has dramatically jeopardised the legacy tools. Recent advances in Deep Learning offer a wide variety of architectures that seem relevant for this purpose. These architectures are based on different data representations as input of their classification process. In this paper, we show the need for a deeper understanding of the features used by Deep Learning models to perform such classification. Our objective is to exploit this knowledge for defining a better data processing so that the chosen architecture will significantly improve the classification process. We will show that some information carried by packet headers need to be analyzed through a separate process. This analysis highlight that current Deep Learning approaches in the literature fail to classify encrypted flows in practice. We therefore propose a new modular Deep Learning architecture called Servername Protocol Packet Network (SPPNet) to overcome this drawback. We will show by a proof of concept that SPPNet allows to perform real-time network flow classification at packet level.**

***Index Terms*—Deep Learning, Encrypted, Network traffic classification**

I. Introduction

From conception to exploitation, traffic characterization plays a fundamental part in the whole life of a network. Simulation models implemented to design network elements could not be relevant without suitable traffic models based on such a characterization. The network setup heavily depends on a good traffic prediction leading to a suitable deployment, and Quality of Service (QoS) enforcement as well as traffic engineering could be challenged by a weak knowledge of the streams actually transported. Intrusion Detection Systems can be based on traffic profiles to detect different types of attacks. Network operators need to be able to classify their traffic in order to implement traffic mitigation, prioritization, ...

Classification is the very first and most salient part of traffic characterization. It can be implemented on a stream basis or at the packet level. A stream based classification requires a state-full implementation. It can use global information, such as inter-arrival, to spot a whole stream and thus classify the corresponding packets [1]. A packet level classification is a sate-less process based only on the information carried by a single packet. In this paper, we will focus on the latter, as we believe that only such a classification can be implemented in real time.

Several techniques of packet level classification have been implemented through decades for Internet Protocol (IP) networks. Port-based techniques are not suitable anymore, because of port obfuscation. Payload-based solutions, such as Deep Packet Inspection (DPI), can be complex to maintain and have proven to be effective as long as the data is not encrypted [2], [3].

In order to cope with header obfuscation or even encryption of data streams, Machine Learning techniques have been introduced such as K-Nearest Neighbors (KNN) [4] or Naives Bayes (NB) [5]. However, the choice of features is important and can greatly impact the performance of the final model and its ability to generalize. In addition, encryption can offend a number of features and thus reduce performance.

Deep Learning methods appear as a mean to automatically and hierarchically extract the relevant features for classification. They are now state-of-the-art methods for classification of encrypted network traffic. Among the methods based on the packet content, we find mainly convolution architectures [6]–[9]. Some use recurrent and convolution architectures to exploit the sequential features existing within the packets of a flow [10], [11]. Some use only recurrent architecture [12] or convolution architecture derivatives such as text convolution [13].

The main challenge with such techniques is to understand the impact of each feature in the data used by the Deep Learning model. As a consequence, the performance and the generalization capacities of a system cannot be assessed easily.

The paper will be organize as follows. In section II, the data collection, and pre-processing perform are presented. In section III, different modeling hypotheses, Deep Learning architectures are presented in order to determine the biases in the classification and to select the best configuration. We believe that headers and payload should be treated separately as they obviously carry different kinds of information. We will show in this paper that the header removal will increase the classification performance. In section IV, from the study done in section III, we will keep in the headers pieces of information define as relevant (port numbers, protocols, ...). We will then use these two information sources separately (payload and headers fields) for an efficient packet classification throw a new architecture named Servername Protocol Packet Network (SPPNet) which improves the performance of the state-of-the-

978-1-7281-8104-2/21/$31.00 ©2021 IEEE

art. Finally, the proposed model is implemented in order to perform real-time classification.

II. Data processing

This section describes the data and the processing done in order to perform state-less classification with Deep Learning models. As already stated, our first aim is to determine the best representation of the data. For this, we need to perform multiple processing on the data set.

A. Collection

The data used for the experiments come from two distinct sources. The first source includes two open access data sets used as reference in the field of classification of encrypted network flows : ISCXVPN2016 [14] and ISCXTor2016 [15]. These data sets present a diversity of applications with a large number of packets. We will use two subsets from these sources : a data-set collected in a TOR (The Onion Router) network, and a data-set of packets encrypted with TLS. The second source includes data collected in our laboratory using the same configuration as for the ISCXVPN2016 [14] and ISCXTor2016 [15] data collection.

Transport Layer Security (TLS) and The Onion Router (TOR) data come from ISCX data sets is called "reference data set" and is used for training and partial evaluation of the realized models. The TLS and TOR data collected in our laboratory form a data set called "our data set" and is used to evaluate the models. This use of various sources allows us to verify the generalisation capabilities of our classification tool.

B. Labellisation

We define seven classes of packet : Chat, E-mail, File Transfer, Peer-To-Peer (P2P), Voice Over IP (VoIP), Streaming, Web browsing. The purpose of a classification tool is thus to classify packet in one of these classes. Such a class definition leads to an application diversity within each class. This will challenge the generalization capacity of the models ; more specifically its ability to face new applications. Table I represents the list of applications present in reference data and our data sets.

C. Pre-processing

Several levels of pre-processing are performed. A first level consisted in filtering the original data. A second level is focused on data representations and formats. Finally, the last level prepared the data for the learning step of Deep Learning models.

For the first level of pre-processing, filtering operations are used to removed information which can caused a bias. Connection initialization packets Transmission Control Protocol (TCP), TLS, Quick UDP Internet Connections (QUIC) are deleted. As shown in [16] some fields in these packets headers can overly facilitate or bias the classification. Some works do not remove them [6]–[11], [13]. We will show during experiment that some headers fields can introduces a bias using a convolution hypothesis for encrypted data.

The second level of pre-processing comes from the observation that no comparison between these different representations and formats has been made in the literature. We chose to jointly study two formats and two representations of the data listed in the table II. Stuffing bytes are used to get uniform sizes.

TABLE I
Application list in each class for each group of both data set.

Classes	Groups	Our data set	Reference data set
Chat	TLS	Facebook, Telegram	Gmail, Hangout, Skype
	TOR	qTox	Facebook, ICQ, AIM
Email	TLS	Outlook, Gmail	Gmail
	TOR	GMX Caramail	
File transfer	TLS	SSH, Google Drive	Facebook, Skype, FTP/SFTP Fillezilla
	TOR	DropBox	
P2P	TLS	Deluge	uTorrent
	TOR	Vuze	
Streaming	TLS	Youtube, Dailymotion	Vimeo, Youtube, Netflix
	TOR		
VoIP	TLS	Facebook	Facebook, Hangout, Skype, VoIPBuster
	TOR	qTox	
Web browsing	TLS	Mozilla Firefox	Firefox, Chrome
	TOR	Tor Browser	Tor Browser

TABLE II
List of data representation, format and shape studied.

Representations	Formats	Values range	Shapes
2D	Integer	0 - 255	40×40
	Bit	0 - 1	111×111
1D	Integer	0 - 255	1536×1
	Bit	0 - 1	12288×1

For the last level of pre-processing, the reference data set is sub-sampled with equity between classes and applications to ensure representativeness. A total of 81003 different packets is used.

The training data corresponds to 80% of the sub-sampled reference data. Validation and test data respectively share 10% of the sub-sampled reference data set. The smaller packets are removed in order to facilitate the training process. Obviously, such packets contain little or no data, so they could be classified based on theirs headers only. This would make sense in a state-full scheme, not in this study.

The evaluation of the models is carried out on the test data and on 21000 packets under-sampled from our data set. These 21000 packets are of various sizes between 0 and 1536 bytes and use applications not present in training data.

III. State-less packet level classification with Deep Learning

This section describes the experiments performed in order to determine jointly the best representation of the data and the best modeling hypothesis. Experiments will show the limitations that exist in current Deep Learning approaches for state-less packet level classification. The conclusions of this analysis will lead us to the definition of a new architecture named SPPNet.

A. Deep Learning architecture

Three different hypotheses are defined on the features present in the packets allowing the classification. Each hypothesis requires the implementation of different Deep Learning architectures. The architectures used as well as their associated assumptions are as follows : (i) a convolution ResNet18 architecture is focus in invariant translation features within data using learned convolution filters, (ii) a ResNet18 architecture with attention [17] learns the invariant features in translation thanks to learned convolution filters as well as their relative position between them. Indeed, a packet respects a form of encapsulation that includes contextual information. This information is relevant for image classification [17]. A reference model using a simple (iii) Euclidean distance allows to verify the relevance of the obtained results with Deep Learning approaches. Subsequently, for each validated hypothesis the best modeling hypothesis will be studied on all types of data format and representation listed in table II.

B. Results and analysis

1) Accuracy: The classification performances of the different models in function of different parameters and data set are respectively listed in the table III for TLS data. We should notice that we have also considered a simple recursive architecture composed of Gated Recurrent Network (GRU) cells [18] by assuming that the features present in the data are sequential but this architecture does not converge. One explanation could be the length of the input data and the lack of a sequential pattern caused by the encryption.

Models learned from the data without removing the headers have good results on test data and are consistent with what is observed in the literature. The packet length has no effect on classification performance which is abnormal. There is a very large gap between test data and our data reflecting a lack of generalization, for all models, independently of the data representation. Applications that are not present in the training data are misclassified. For TOR data, the place where the data is taken has no influence on the classification performances. The difference of performances compared to TLS data can be explained by the effect of a more robust encryption and packet size obfuscation.

After removing the header IP the findings remain similar. However, the removal of header TCP/UDP improves the generalization performance. On the data TOR there is no increase in classification performance on our data (similar to random). As soon as the headers IP and TCP/UDP are removed, as the packet size on our data increases, so does the performance. In models using data TLS applications not present in the training data are correctly classified. Classes containing few or no encrypted packets are not better classified than other classes. Models using an attention architecture are more prone to over fitting and lack of generalization despite the removal of headers.

2) Interpretation: An interpretation in the input space with Gradient Class Activation Map (GradCAM) [19] and the Occlusion Map [20] shows the limits of such models.

GradCAM [19] is applied on the last convolution layer after the non-linearity. This technique makes it possible to see where the convolutional filter reacts to the input data in order to perform classification. Warm colors mean a strong reaction. All the models focus on the headers for the classification. Nevertheless, there is a slight focus on the packet content which may explain the results slightly higher than the random and reference models. After removing the IP header the focus becomes more pronounced and more localized. On the other hand, removing both the IP header and TCP/UDP allows a more diffuse and less localized focus on the headers. This focus shows a better ability to generate models. This is less noticeable on the TOR data and can be explained by the use of stuffing bits which forces the model to focus on less noisy elements located at the beginning of the packets. The implementation of GradCAM [19] on seven packets of different classes is visible in figure 1 for the TLS test data. For the TOR data the implementation of GradCAM is visible in figure 2. GradCAM shows similar results on integer format.

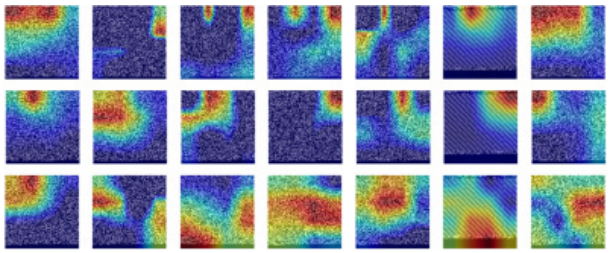

Fig. 1. GradCAM [19] applied on seven packets in two dimensions representation in bit format from TLS group on test data. Each lines represent a level of packet suppression and each columns a packet in class from left to right : Chat, E-mail, File Transfer, P2P, Streaming, VoIP, Web Browsing.

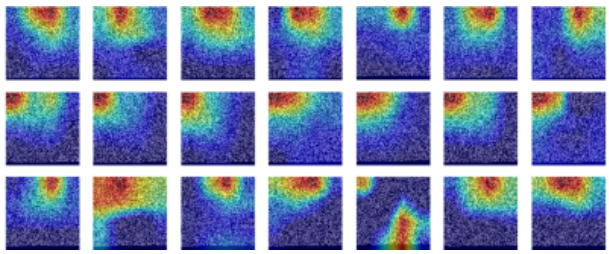

Fig. 2. GradCAM [19] applied on seven packets in two dimensions representation in bit format from TOR group on test data. Each lines represent a level of packet suppression and each columns a packet in class from left to right : Chat, E-mail, File Transfer, P2P, Streaming, VoIP, Web Browsing.

Occlusion Map [20] are also applied with a 2×2 filter for integer packets and 3×3 for binary packets. The goal is to hide each byte of a packet and see how the probability of the right class associated with this packet assigned by the output model evolves. Before removing headers, the models focus on bytes belonging to the IP source, IP destination, port source, port destination and sometimes other fields like sequence number. This focus explains the lack of generalization. After removing the headers, the models focus on more global structure located

TABLE III

ACCURACY OF MODELS ACCORDING TO FORMAT, REPRESENTATION AND LEVEL OF HEADER DELETION FOR TLS DATA : IP (LAYER 3), THEN IP AND USER DATAGRAM PROTOCOL (UDP)/TCP (LAYER 3/LAYER 4). BEST PERFORMANCE FOR EACH DATA SET AND HEADER REMOVAL ARE **BOLDED**.

Data	Models	Type	Test	Our	Test L3	Our L3	Test L3/L4	Our L3/L4
TLS	ResNet18 1D	Integer	0.999	**0.381**	0.975	0.422	0.589	**0.502**
		Bit	**0.999**	0.285	**0.997**	**0.460**	0.626	0.273
	ResNet18 2D	Integer	0.991	0.278	0.945	0.341	0.636	0.405
		Bit	0.996	0.235	0.992	0.417	0.703	0.418
	ResNet18 2D Attention	Integer	-	-	-	-	**0.997**	0.460
		Bit	-	-	-	-	-	-
	L2 Distance	Integer	0.466	0.231	0.413	0.233	0.397	0.267
		Bit	0.535	0.235	0.470	0.220	0.395	0.266

inside the packet but which cannot be interpreted because it is encrypted. The implementation of Occlusion Map on seven packets of different classes in test data from TLS data set is shown in figure 3. The results are similar on the TOR data but the focus remains slightly more pronounced on the headers after removing them unlike the TLS data.

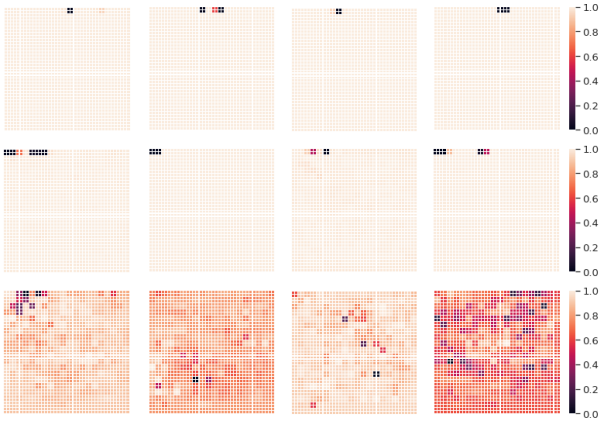

Fig. 3. Occlusion Map [20] applied on packets in two dimensions representation in integer format from TLS group. Each line represents a level of packet suppression and each column a random packet extracted from data.

Removing headers prevents models from focusing on features that cannot be generalized. Indeed, the convolution models assume the hypothesis of invariant patterns in translation present in the data. However, the packet structures of a flow contain multiple pieces of information requiring different modeling assumptions. For example, raw text cannot be treated as a simple pattern. This observation explains the improvement in the performance of models using the first packets of a flow [8], [11], [13]. Indeed, the initialization of a TLS connection includes information such as the name of the server which is a pattern to determine the type of application. However, this pattern is not generalizable and is specific to the applications present in the training data. For example, if we want to classify application, if the field TLS server name change the models will not be able to associate the packet to the right application. This may perhaps explains the improved performance obtained with text convolution that is better able to detect this type of structure [13].

C. Conclusion

Our experiments have shown that one dimensional representation in integer format with a simple convolutional architecture offers the best classification performance. Headers cause a bias and make current approaches using convolutional architectures [6]–[11], [13] not accurate in practice. Nevertheless, some information contained in the headers should be exploitable. To do so, it is necessary to define specific assumptions and processing for each exploited information.

IV. SPPNET ARCHITECTURE

The previous study leads us to propose a new architecture, called SPPNet which stands for Servername Protocol Packet Network. Its approach aims at exploiting all the available features in the data sets, identified as generalizable :

- **The packet** without headers IP and TCP/UDP.
- **The type of protocol used**, UDP or TCP. The traffic UDP corresponds to a traffic linked to VoIP and possibly P2P.
- **The protocol encapsulated by UDP or TCP** is identified from the ports. For example, knowing that a packet carries Post Office Protocol over SSL (POP3S) can help classification for Email packets.
- **The domain name** associated with the source or destination IP address of the packets. For example, “vesta.web.telegram.org” can help to determine if it is a Chat class packet from the Telegram application.
- **The server name** present when initializing the TLS connection if it is present. For example, “outlook.office365.com” can help the model to classify the packet in the Email class.

The SPPNet architecture is divided into two parts, as shown in Figure 4: the extraction of semantic feature from each source (first part) and theirs combinations (second part).

The first part is composed of:

- **ResNet18** which will take packets in 1D representation in integer format as input for performance reasons observed in table III. The output is a vector of 512 dimensions.
- **Recurrent layer** of type GRU [18], composed of projection blocks that will take the name of the server or the domain name as input. The output of the recurrent block projects the data in a five-dimensional vector.

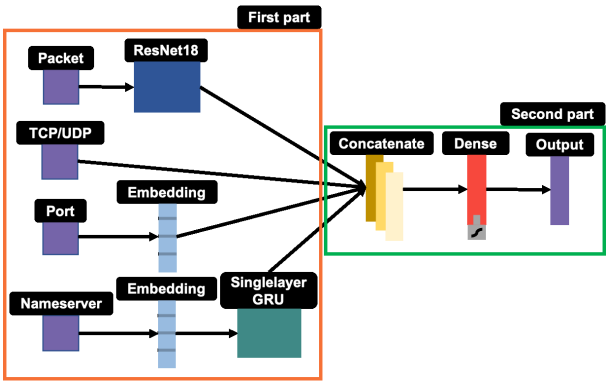

Fig. 4. SPPNet architecture diagram.

- **Embedding layer** which will take as input the name of the protocol associated with the port. The embedding layer embeds the data into a two-dimensional vector.
- **Input layer** which will take as input the type of transport protocol used (TCP or UDP). The output is a value of one dimension.

The output dimension of the embedding layers is set to be the fourth root of the corpus size [21]. The outputs of each layer and architecture are then concatenated and connected to a new fully connected layer common to all.

1) Pre-processing: For packets, the Ethernet headers, IP and TCP/UDP have been removed to avoid bad generalization. On the other hand, all packet sizes have been kept for experiments.

For server names, they are extracted from the connections TLS and QUIC by retrieving the server name field when it is present. Recording Canonical NAME (CNAME) of Domain Name System (DNS) messages before setting up a connection is also used. An association table server name, source/destination address, source/destination ports is used to associate a server name with all the concerned packets. In the case where a packet has both a domain name and a server name, the domain name is chosen in priority in order to guarantee a good generalization.

Server names are pre-processed for learning step. In the case of the server name "vesta.web.telegram.org", the processing is done as follows:

- **Step 1**: The dots are deleted giving the input: "vesta web telegram org"
- **Step 2**: The order of the words is inverted giving the input: "org telegram web vesta"

These operations remains considering server names as a sentence. Moreover, by reversing the order of the sentence, we can encode the importance of each server in the hierarchy of server names. The server name "org" being more important, higher in the hierarchy, it is at the beginning of the sentence. Another interest is to learn how to distinguish the different traffic classes in the same application.

2) Experiments: The SPPNet architecture allows the extraction of semantic features related to the combination of these features. Contrary to the simple convolution product, we are interested in the structure of the server names and not in their simple presence or absence as in previous works where we use the concatenation of the first packets of a flow to initialize a connection [8], [11], [13].

Learning is done in two stages. Each layer or architecture of the first part becomes an independently learned model in a supervised manner. Each layer or architecture is connected to a completely connected block during the learning process. After learning, each layer or architecture is retrieved with these weights and disconnected from their completely connected block. The outputs of the different architecture layer are then concatenated and connected to a new fully connected layer common to all. During the second learning phase, only the fully connected block is learned so that it weights the importance of the outputs of each layer with respect to the final classification.

Two sets of data are used for training in each phase. They underwent the same division as in previous experiments and include 81003 packets sampled in the same way. Their role is as follows:

- **"local" data set** : used for training the models belonging to the first part of the architecture. It includes packets ranging from 768 to 1536 bytes.
- **"global" data set** : used for training the final model. It includes packets which not exist in "local" data set. Packet sizes ranging from 0 to 1536 bytes and follow an uniform distribution.

The advantage of using different data sets is to avoid that the final model layer focuses unfairly on the input models. Another interest is to allow the model to integrate the fact that the ResNet18 model performs poorly on small packets but that it should not focus entirely on the model integrating the server names. Indeed, the model may sometimes perform worse on large packets compared to the ResNet18 model, and the final model should also incorporate the lack of information for some packets. Some packets have no associated server names or no identified protocol. The "local" and "global" data sets are also designed to maximize this diversity. Each data set is divided into three subsets (learning, test and validation) as above. As the classes are perfectly balanced the cross-entropy categorical loss function is used.

3) Results: The results of the models based on the combination of test and our data input information on known and unknown applications are listed in the table IV. Considering all the additional information improves the accuracy from 5.1% to 14.8% on our data set of TLS network according to table III and perform state-of-art performance due to the lack of generalization of others methods as shown in section III. The low number of domain names (2000 different server names) has an impact on the generalization of the models. Applications that are not present in the training data are less well classified when server names are present. Packet size has no influence on classification performance. On the other hand, small packets are better classified with these additional information.

TABLE IV
RESULT OF THE COMBINATION OF SEVERAL SOURCES OF INFORMATION IN SPPNET.

Combinations	Test	Our	Know app.	Unk. app.
Packet/Name server	0.670	0.553	0.854	0.211
Packet/Port	0.589	0.596	0.675	**0.581**
Packet/Protocol	0.356	0.310	0.318	0.344
Packet/Port/ Protocol	0.638	0.622	0.761	0.570
Packet/Port/ Protocol/Name server	**0.857**	**0.650**	**0.880**	0.484
Packet/Protocol/ Nameserver	0.720	0.539	0.875	0.177

However, these performances are about 10% lower on the test data than the state of the art approaches based on flow statistics [22] which cannot classify in real-time. Moreover, the evaluation of the models based on the flow statistics is not performed on two different data sets, there is no guarantee of generalization. Nevertheless, adding information about the flow statistics to which the packet belongs as input to SPPNet could increase the classification performance.

A. Proof of concept

SPPNet is trained using a total of 237,636 packets from reference and our data sets. It is implemented on a laptop computer with Python 3. The source code is available [1] with a visualization tool in Javascript to see the classification in real-time.

V. CONCLUSION

In this paper, we have studied Deep Learning architectures for state-less classification of encrypted traffic. We have shown that, without any pre-processing, the models may focus on features that could not be generalized.

To overcome this limitation, we first have processed headers and data separately. Then, we have defined the different sources of information and studied them independently in order to allow good generalization performances.

Finally, we have proposed a new modular architecture, called SPPNet, that implements this approach and allows, using packet-level classification, a real-time classification. Its implementation combined with a flow table has achieved better performance by taking into account the notion of flow. This approach offers modularity by defining new sources of information as shown on an implemented proof of concept.

A state-full version of SPPNet can now be considered. Statistics such as correlations between packets could help increasing performance. However, we need to determine whether this gain is worth the loss of real-time classification.

As a next step, we plan to study the impact of encryption and features within the encrypted data. Whatever the information used by the Deep Learning architecture to carry on classification is, it could be used to implement side-channel attacks.

[1] Source code : https://github.com/fmeslet/SPPNet

REFERENCES

[1] S. Rezaei and X. Liu, "Deep Learning for Encrypted Traffic Classification: An Overview," *IEEE Commun. Mag.*, vol. 57, no. 5, pp. 76–81, May 2019.

[2] S. H. Yeganeh, M. Eftekhar, Y. Ganjali, R. Keralapura, and A. Nucci, "CUTE: Traffic Classification Using TErms," in *21st ICCCN*. Munich, Germany: IEEE, Jul. 2012, pp. 1–9.

[3] S. Sen, O. Spatscheck, and D. Wang, "Accurate, scalable in-network identification of p2p traffic using application signatures," in *13th WWW '04*. New York, NY, USA: ACM Press, 2004, p. 512.

[4] R. Bar Yanai, M. Langberg, D. Peleg, and L. Roditty, "Realtime Classification for Encrypted Traffic," in *Experimental Algorithms*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2010, vol. 6049, pp. 373–385, series Title: Lecture Notes in Computer Science.

[5] D. McGaughey, T. Semeniuk, R. Smith, and S. Knight, "A systematic approach of feature selection for encrypted network traffic classification," in *SysCon*. Vancouver, BC: IEEE, Apr. 2018, pp. 1–8.

[6] M. Lotfollahi, M. Jafari Siavoshani, R. Shirali Hossein Zade, and M. Saberian, "Deep packet: a novel approach for encrypted traffic classification using deep learning," *Soft Comput*, vol. 24, no. 3, pp. 1999–2012, Feb. 2020.

[7] Z. Chen, K. He, J. Li, and Y. Geng, "Seq2Img: A sequence-to-image based approach towards IP traffic classification using convolutional neural networks," in *Big Data*. Boston, MA: IEEE, Dec. 2017, pp. 1271–1276.

[8] W. Wang, M. Zhu, J. Wang, X. Zeng, and Z. Yang, "End-to-end encrypted traffic classification with one-dimensional convolution neural networks," in *ISI*. Beijing, China: IEEE, Jul. 2017, pp. 43–48.

[9] F. Pacheco, E. Exposito, and M. Gineste, "A framework to classify heterogeneous Internet traffic with Machine Learning and Deep Learning techniques for satellite communications," *Computer Networks*, vol. 173, p. 107213, May 2020.

[10] P. Wang, F. Ye, X. Chen, and Y. Qian, "Datanet: Deep Learning Based Encrypted Network Traffic Classification in SDN Home Gateway," *IEEE Access*, vol. 6, pp. 55 380–55 391, 2018.

[11] Z. Zou, J. Ge, H. Zheng, Y. Wu, C. Han, and Z. Yao, "Encrypted Traffic Classification with a Convolutional Long Short-Term Memory Neural Network," in *HPCC/SmartCity/DSS*. Exeter, United Kingdom: IEEE, Jun. 2018, pp. 329–334.

[12] C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li, "FS-Net: A Flow Sequence Network For Encrypted Traffic Classification," in *INFOCOM 2019*. Paris, France: IEEE, Apr. 2019, pp. 1171–1179.

[13] M. Song, J. Ran, and S. Li, "Encrypted Traffic Classification Based on Text Convolution Neural Networks," in *ICCSNT*. Dalian, China: IEEE, Oct. 2019, pp. 432–436.

[14] G. Draper-Gil, A. H. Lashkari, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of Encrypted and VPN Traffic using Time-related Features:," in *2nd ICISSP*. Rome, Italy: SCITEPRESS - Science and and Technology Publications, 2016, pp. 407–414.

[15] A. Habibi Lashkari, G. Draper Gil, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of Tor Traffic using Time based Features:," in *3rd ICISSP*. Porto, Portugal: SCITEPRESS - Science and Technology Publications, 2017, pp. 253–262.

[16] S. Rezaei, B. Kroencke, and X. Liu, "Large-Scale Mobile App Identification Using Deep Learning," *IEEE Access*, vol. 8, pp. 348–362, 2020.

[17] I. Bello, B. Zoph, Q. Le, A. Vaswani, and J. Shlens, "Attention Augmented Convolutional Networks," in *CVF/ICCV*. Seoul, Korea (South): IEEE, Oct. 2019, pp. 3285–3294.

[18] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling," *arXiv:1412.3555 [cs]*, Dec. 2014, arXiv: 1412.3555.

[19] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization," in *ICCV*. Venice: IEEE, Oct. 2017, pp. 618–626.

[20] K. Heinrich, P. Zschech, T. Skouti, J. Griebenow, and S. Riechert, "Demystifying the Black Box: A Classification Scheme for Interpretation and Visualization of Deep Intelligent Systems," p. 11.

[21] T. Team. (2017) Introducing tensorflow feature columns. [Online]. Available: https://developers.googleblog.com/2017/11/introducing-tensorflow-feature-columns.html

[22] S. Rezaei and X. Liu, "Multitask Learning for Network Traffic Classification," in *ICCCN*. Honolulu, HI, USA: IEEE, Aug. 2020, pp. 1–9.